

PySpark from scratch

● LEVEL 1 — DataFrame Basics

1. Read the employees table into a DataFrame.

```
df_emp = spark.table("ashish.employee.employees")
```

LinkedIn@AshishZope

2. Display the complete employee DataFrame.

```
display(df_emp)  
df_emp.show(10) # 10 in the show() method is the number of rows to display
```

LinkedIn@AshishZope

3. Display the first 10 employees.

```
display(df_emp.limit(10))
```

LinkedIn@AshishZope

4. Display the first 20 employees without truncating string values.

```
df_emp.show(20, truncate=False)
```

LinkedIn@AshishZope

5. Print the schema of the employee DataFrame.

```
df_emp.schema  
df_emp.printSchema()
```

LinkedIn@AshishZope

6. Print all column names.

LinkedIn@AshishZope

```
print(df_emp.columns)
df_emp.columns
```

7. Count the total number of employees.

LinkedIn@AshishZope

```
df_emp.count()
```

8. Display only the first employee.

LinkedIn@AshishZope

```
display(df_emp.limit(1))
```

9. Display 5 employees using `take()`.

LinkedIn@AshishZope

```
df_emp.take(5)
```

10. Check the number of partitions of the DataFrame.

LinkedIn@AshishZope

```
df_emp.rdd.getNumPartitions()
```

11. Display employees using `display()`.

LinkedIn@AshishZope

```
display(df_emp)
```

12. Display employees using `show()`.

LinkedIn@AshishZope

```
df_emp.show()
```

13. Display the DataFrame vertically.

LinkedIn@AshishZope

```
df_emp.show(vertical=True)
```

14. Create a temporary view called `employees_view`.

LinkedIn@AshishZope

```
df_emp.createOrReplaceTempView("employees_view")
```

15. Read the temporary view using Spark SQL.

LinkedIn@AshishZope

```
df_emp_sql = spark.sql("SELECT * FROM employees_view")
```

● LEVEL 2 — SELECT Operations

16. Select only: `employee_id`, `first_name`, `last_name`, `salary`

LinkedIn@AshishZope

```
df_emp.select("employee_id", "first_name", "last_name", "salary").show()
```

17. Select employee ID, name and department.

LinkedIn@AshishZope

```
df_emp.join(df_dept, df_emp.department_id == df_dept.department_id,
"inner").select("employee_id", "first_name", "last_name",
"department_name").show()
```

18. Select employee ID, email and phone.

LinkedIn@AshishZope

```
display(df_emp.select("employee_id", "email", "phone"))
```

19. Rename **first_name** as **employee_first_name**.

LinkedIn@AshishZope

```
df_emp.selectExpr("first_name AS employee_first_name").show()
```

20. Rename **salary** as **annual_salary**.

LinkedIn@AshishZope

```
df_emp.selectExpr("salary AS annual_salary").show()
```

21. Select salary and bonus.

LinkedIn@AshishZope

```
df_emp.select("salary", "bonus").show()
```

22. Select employee information and rename multiple columns.

LinkedIn@AshishZope

```
df_emp.selectExpr(  
    "first_name || ' ' || last_name as employee_full_name",  
    "salary + bonus as total_compensation",  
    "salary + bonus > 1000000 as is_highly_paid"  
) .show()
```

23. Select all columns except **phone**.

LinkedIn@AshishZope

```
df_emp.drop("phone").show()
```

24. Select only: **text**, **employee_id**, **first_name**, **last_name**, **city**, **state**

LinkedIn@AshishZope

```
df_emp.select("employee_id", "first_name", "last_name", "city",  
"state").show()
```

25. Select employee information and create an alias for salary.

LinkedIn@AshishZope

```
df_emp.selectExpr("first_name AS f_name", "salary AS salary_new").show()
```



LEVEL 3 — FILTER Operations

26. Find all employees whose salary is greater than 1,000,000.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.salary > 1000000).show()  
df_emp.select("employee_id", "first_name", "last_name", "city", "state",  
"salary").filter("salary > 1000000").show()
```

27. Find employees whose salary is less than 700,000.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.salary < 700000).show()
```

28. Find employees whose salary is between 700,000 and 1,500,000.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.salary >= 700000) & (df_emp.salary <= 1500000)).show()  
df_emp.filter("salary > 700000 AND salary < 1500000").show()
```

29. Find employees from Pune.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.city == "Pune").show
```

30. Find employees from Mumbai.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.city == "Mumbai").show()
```

31. Find employees from Pune or Mumbai.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.city == "Pune") | (df_emp.city == "Mumbai")).show()  
df_emp.filter("city == 'Mumbai' OR city == 'Pune'").show()  
df_emp.filter("city IN ('Mumbai', 'Pune')").show()
```

32. Find employees who are currently active.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.employment_status == "Active").show()
```

33. Find inactive employees.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.employment_status == "Inactive").show()
```

34. Find employees whose gender is Male.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.gender == "Male").show()
```

35. Find employees whose salary is greater than 1,000,000 and who are active.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.salary > 1000000) & (df_emp.employment_status ==  
"Active")).show()  
df_emp.filter("salary > 1000000 AND employment_status == 'Active'").show()
```

36. Find employees from Pune with salary greater than 1,000,000.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.city == "Pune") & (df_emp.salary > 1000000)).show()  
df_emp.filter("salary > 1000000 AND city == 'Pune'").show()
```

37. Find employees who are not from Pune.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.city != "Pune").show()  
df_emp.filter("city != 'Pune'").show()
```

38. Find employees whose department ID is 1.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.department_id == 1).show()
```

39. Find employees whose job ID is 2 or 3.

LinkedIn@AshishZope

```
df_emp.filter((df_emp.job_id == 2) | (df_emp.job_id == 3)).show()  
df_emp.filter("job_id IN (2, 3)").show()  
df_emp.filter(df_emp.job_id.isin(2,3)).show()
```

40. Find employees whose manager ID is NULL.

LinkedIn@AshishZope

```
df_emp.filter(df_emp.manager_id.isNull()).show()
```

● LEVEL 4 — Computed Columns

41. Create **annual_salary** from salary.

LinkedIn@AshishZope

```
df_emp.withColumn("annual_salary", df_emp.salary * 12).show()
```

42. Create **increased_salary** with a 10% increment.

LinkedIn@AshishZope

```
df_emp.withColumn(
    "salary_10_percent",
    col("salary") * 10 / 100
).select(
    col("first_name").alias("f_name"),
    col("salary"),
    col("salary_10_percent").alias("increased_salary")
).show()
```

43. Create **bonus_amount** as 15% of salary.

DATA ENGINEER

LinkedIn@AshishZope

```
df_emp.withColumn("bonus_amount", df_emp.salary * 15 / 100).select(
    col("first_name").alias("f_name"),
    col("salary"),
    col("bonus_amount").alias("bonus").cast("int")
).show()
```

44. Create **total_compensation: salary + bonus**

LinkedIn@AshishZope

```
df_emp.withColumn("total_compensation", df_emp.salary + df_emp.bonus).select(
    col("first_name").alias("f_name"),
    col("salary"),
    col("bonus"),
    col("total_compensation")
).show()
```

45. Create **double_salary**.

LinkedIn@AshishZope

```
df_emp.withColumn("double_salary", col("salary") * 2).select("first_name",
"double_salary").show(5)
```

46. Create **salary_after_5_percent_increment**.

LinkedIn@AshishZope

```
df_emp.withColumn("salary_after_5_percent_increment", col("salary") *
1.05).select("first_name", "salary_after_5_percent_increment").show(5)
```

47. Create **salary_difference** between salary and bonus.

LinkedIn@AshishZope

```
df_emp.withColumn("salary_difference", col("salary") -
col("bonus")).select("first_name", "salary", "bonus",
"salary_difference").show(5)
```

48. Create **monthly_salary** from annual salary.

LinkedIn@AshishZope

```
df_emp.withColumn("monthly_salary", col("salary") / 12) \
    .select("first_name", col("monthly_salary").cast("int")) \
    .show(5)
```

49. Create **monthly_bonus**.

LinkedIn@AshishZope

```
df_emp.withColumn("monthly_bonus", col("bonus") / 12) \
      .select("first_name", col("monthly_bonus").cast("int")) \
      .show(5)
```

50. Create salary_category:

LinkedIn@AshishZope

```
< 700000      → Low
700000-1200000 → Medium
1200000-2000000 → High
> 2000000      → Very High
```

LinkedIn@AshishZope

```
df_emp.selectExpr(
  "first_name",
  """CASE
    WHEN salary >= 2000000 THEN 'Very High'
    WHEN salary >= 700000 AND salary < 2000000 THEN 'High'
    WHEN salary < 700000 THEN 'Low'
    ELSE 'Unknown'
  END AS salary_tier"""
).show()
```

51. Create employee_type based on employment status.

52. Create a column indicating whether salary is above 1 million:

LinkedIn@AshishZope

```
Yes / No
```

53. Create manager_status:

LinkedIn@AshishZope

```
Manager Assigned
No Manager
```

54. Create full_name.

55. Create `email_domain`.



● LEVEL 5 — String Operations

56. Convert first names to uppercase.

57. Convert last names to lowercase.

58. Create full name using first and last name.

59. Extract the first 3 characters of first name.

60. Extract the last 3 characters of last name.

61. Find the length of employee names.

62. Remove spaces from employee names.

63. Replace part of an email address.

64. Extract email domain.

65. Check whether email contains `@`.

66. Find employees whose first name starts with `A`.

67. Find employees whose last name ends with `a`.

68. Create an employee username:

```
firstname.lastname
```

LinkedIn@AshishZope

69. Create an employee code:

```
EMP_<employee_id>
```

LinkedIn@AshishZope

70. Create a display name:

```
LASTNAME, FIRSTNAME
```

LinkedIn@AshishZope



LEVEL 6 — NULL Handling

71. Find employees whose manager ID is NULL.

72. Count NULL values in every column.

73. Replace NULL city with **Unknown.**

74. Replace NULL salary with **0.**

75. Replace NULL bonus with **0.**

76. Replace NULL manager ID with **-1.**

77. Create a column showing whether manager ID is NULL.

78. Find employees with missing phone numbers.

79. Find employees with missing email addresses.

80. Remove rows where employee ID is NULL.

● LEVEL 7 — Sorting

81. Sort employees by salary ascending.
82. Sort employees by salary descending.
83. Sort employees by first name.
84. Sort employees by department and salary.
85. Find the highest-paid employee.
86. Find the lowest-paid employee.
87. Find the top 5 highest-paid employees.
88. Find the bottom 5 employees by salary.
89. Sort by salary descending and bonus ascending.
90. Sort employees by hire date.

● LEVEL 8 — DISTINCT & DUPLICATES

91. Find distinct cities.
92. Find distinct states.
93. Find distinct departments.

- 94. Count distinct cities.
- 95. Count distinct departments.
- 96. Find duplicate employee IDs.
- 97. Find duplicate email addresses.
- 98. Remove duplicate employees based on employee ID.
- 99. Remove duplicate records based on email.
- 100. Keep the latest record when duplicate employee IDs exist.



LEVEL 9 — Aggregations

- 101. Find total salary of all employees.
- 102. Find average salary.
- 103. Find minimum salary.
- 104. Find maximum salary.
- 105. Find total bonus.
- 106. Count employees.
- 107. Count employees by department.
- 108. Find average salary by department.
- 109. Find maximum salary by department.

- 110. Find minimum salary by department.
- 111. Find total salary by department.
- 112. Find average bonus by department.
- 113. Find employee count by city.
- 114. Find employee count by state.
- 115. Find average salary by city.
- 116. Find departments having more than 5 employees.
- 117. Find departments whose average salary is greater than 1 million.
- 118. Find the department with the highest total salary.
- 119. Find the department with the highest average salary.
- 120. Find the city having the highest employee count.



DATA ENGINEER

LEVEL 10 — GROUP BY + HAVING Scenarios

- 121. Find departments with more than 3 employees.
- 122. Find departments with average salary > ₹10 lakh.
- 123. Find cities with more than 5 employees.
- 124. Find departments where maximum salary > ₹20 lakh.
- 125. Find departments where total salary > ₹50 lakh.

- 126. Find cities where average salary > ₹10 lakh.
- 127. Find job IDs having more than 5 employees.
- 128. Find departments having at least one employee earning > ₹20 lakh.
- 129. Find departments where minimum salary is > ₹5 lakh.
- 130. Find departments with both:

```
employee_count > 5  
average_salary > 10 lakh
```

LinkedIn@AshishZope

● LEVEL 11 — Date Operations

- 131. Find employees hired after 2023-01-01.
- 132. Find employees hired before 2023-01-01.
- 133. Find employees hired during 2023.
- 134. Find employees hired during 2024.
- 135. Extract hire year.
- 136. Extract hire month.
- 137. Extract hire day.
- 138. Calculate employee experience in days.
- 139. Calculate employee experience in years.

140. Categorize employees:

LinkedIn@AshishZope

< 2 years	→ Fresher
2-5 years	→ Experienced
> 5 years	→ Senior

141. Find employees hired in the last 2 years.**142. Find employees hired in the current year.****143. Find employees hired in January.****144. Count employees hired by year.****145. Count employees hired by month.****146. Find the earliest hire date.****147. Find the latest hire date.****148. Find employees with birthdays in a particular month.****149. Calculate age from date of birth.****150. Find employees older than 30.**

LEVEL 12 — JOIN Operations

Load:

```
df_emp  
df_dept  
df_jobs
```

- 151. Inner join employees with departments.**
- 152. Add department name to employees.**
- 153. Add department location to employees.**
- 154. Join employees with jobs.**
- 155. Add job title to employees.**
- 156. Add job level to employees.**
- 157. Add minimum and maximum salary for the job.**
- 158. Join employees + departments + jobs.**
- 159. Find employees whose salary is outside their job's salary range.**
- 160. Find employees earning above the maximum salary of their job.**
- 161. Find employees earning below the minimum salary of their job.**
- 162. Find departments having no employees.**
- 163. Find employees whose department doesn't exist in the department table.**
- 164. Find employees whose job doesn't exist in the jobs table.**
- 165. Perform a left join between employees and departments.**
- 166. Perform a right join.**

167. Perform a full outer join.

168. Perform a cross join and understand the result.



LEVEL 13 — Advanced JOIN Scenarios

169. Find employees and their department heads.

170. Find employees and their managers using a self join.

171. Find employees who report directly to each manager.

172. Find managers who have more than 5 employees.

173. Find employees who don't have a manager.

174. Find departments without a department head.

175. Find employees working on projects.

176. Find employees who have no project.

177. Find employees assigned to multiple projects.

178. Find the number of projects per employee.

179. Find the number of employees per project.

180. Find the department with the most projects.



LEVEL 14 — Window Functions

- 181. Rank employees by salary.
 - 182. Rank employees by salary within each department.
 - 183. Find top 3 employees in each department.
 - 184. Find highest-paid employee in each department.
 - 185. Find second-highest-paid employee in each department.
 - 186. Find third-highest-paid employee in each department.
 - 187. Use `row_number()` to assign numbers to employees.
 - 188. Use `rank()` on salary.
 - 189. Use `dense_rank()` on salary.
 - 190. Compare `row_number`, `rank`, and `dense_rank`.
 - 191. Find employees whose salary is above department average.
 - 192. Find employees whose salary is below department average.
 - 193. Calculate department average salary for every employee.
 - 194. Calculate salary difference from department average.
 - 195. Calculate salary percentage compared to department total.
-



LEVEL 15 — LAG / LEAD

Using `salary_history`:

196. Find previous salary for each employee.
 197. Find next salary for each employee.
 198. Calculate salary increment amount.
 199. Calculate salary increment percentage.
 200. Find employees whose salary increased by more than 10%.
 201. Find employees whose salary decreased.
 202. Find the first salary record for each employee.
 203. Find the latest salary record for each employee.
 204. Find the number of salary changes per employee.
 205. Find the maximum salary ever received by each employee.
-



LEVEL 16 — Attendance Practice

Using:

```
df_attendance
```

LinkedIn@AshishZope

206. Find all absent employees.
207. Find all employees who worked from home.
208. Count Present/Absent/WFH/Leave.

209. Find attendance count by employee.

210. Find absent count by employee.

211. Find WFH count by employee.

212. Calculate attendance percentage.

213. Find employees with attendance below 80%.

214. Find employees with more than 2 absences.

215. Find department-wise attendance percentage.

216. Find the employee with the highest absence count.

217. Find the employee with the highest WFH count.

218. Calculate working hours:

```
check_out - check_in
```

LinkedIn@AshishZope

219. Find employees who worked more than 9 hours.

220. Find employees who arrived after 9:15 AM.

LEVEL 17 — Data Quality

221. Check duplicate employee IDs.

222. Check NULL employee IDs.

223. Check NULL salaries.

224. Find negative salaries.

225. Find employees with salary = 0.

226. Validate email format.

227. Find duplicate emails.

228. Find future hire dates.

229. Find employees whose DOB is after hire date.

230. Find employees whose salary is outside job salary range.

231. Check invalid department IDs.

232. Check invalid job IDs.

233. Create a data-quality status column:

Valid
Invalid

LinkedIn@AshishZope

234. Create separate DataFrames for valid and invalid records.

235. Count invalid records by validation rule.

● LEVEL 18 — Date + Business Scenarios

236. Find employees hired in the last 1 year.

237. Find employees with more than 5 years experience.

238. Find employees completing 1 year this month.

239. Find employees completing 5 years this year.

240. Calculate employee age.

241. Calculate employee tenure.

242. Find average tenure by department.

243. Find oldest employee in each department.

244. Find newest employee in each department.

245. Find monthly hiring trends.



LEVEL 19 — Delta Lake

246. Save employee DataFrame as a Delta table.

247. Read the Delta table.

248. Append new employee records.

249. Overwrite the Delta table.

250. Update employee salary using Delta.

251. Delete inactive employees.

252. Perform Delta **MERGE**.

253. Implement insert + update using MERGE.

254. Implement SCD Type 1.

255. Implement SCD Type 2.

256. Query Delta table history.

257. Perform Delta time travel.

258. Restore a previous version.

259. Add a new column using schema evolution.

260. Optimize a Delta table.



LEVEL 20 — PySpark Performance

261. Check DataFrame partitions.

262. Repartition employee data.

263. Coalesce employee data.

264. Compare `repartition()` and `coalesce()`.

265. Cache a DataFrame.

266. Check whether DataFrame is cached.

267. Unpersist a DataFrame.

268. Use `explain()`.

269. Analyze a slow join.

270. Broadcast the department table.

271. Compare normal join vs broadcast join.

272. Identify a potential data-skew problem.

273. Handle a skewed join.

274. Compare built-in functions with a Python UDF.

275. Explain why `collect()` can be dangerous.



LEVEL 21 — Real Interview Scenarios

276.

You receive 10 million employee records every day. Find duplicate records efficiently.

277.

Your employee DataFrame contains 5 million rows and department contains only 20 rows. How would you optimize the join?

278.

A PySpark job that normally takes 10 minutes suddenly takes 45 minutes. What would you investigate?

279.

Your DataFrame has 2 billion records. How would you avoid collecting data to the driver?

280.

One department contains 80% of all records. How would you handle data skew?

281.

You need to calculate the top 3 salaries per department.

282.

You need the second-highest salary per department, including duplicate salaries.

283.

You receive employee updates daily. Existing employees should be updated and new employees inserted.

284.

You need complete employee salary history. Implement SCD Type 2.

285.

An employee's salary changes from ₹10 lakh to ₹12 lakh. Store both versions.

286.

Your source contains NULL values. Define a data-quality framework.

287.

Your source schema changes by adding a new column. How will your Delta pipeline handle it?

288.

Your source sends the same file twice. How will you make your pipeline idempotent?

289.

A file contains 1 million good records and 10,000 bad records. How will you separate them?

290.

Your pipeline fails halfway through processing. How will you restart it safely?



LEVEL 22 — End-to-End Employee Project

Now combine everything.

291.

Read all employee-related tables.

292.

Create an employee master DataFrame containing:

```
employee_id
full_name
email
department
job_title
salary
bonus
city
hire_date
employment_status
```

LinkedIn@AshishZope

293.

Add:

```
experience_years
age
salary_category
```

LinkedIn@AshishZope

294.

Calculate department-level:

```
employee_count  
average_salary  
minimum_salary  
maximum_salary  
total_salary
```

LinkedIn@AshishZope

295.

Rank employees by salary within department.

296.

Find top 3 employees in each department.

297.

Find employees earning above their department average.

298.

Join employee data with attendance.

299.

Calculate employee attendance percentage.

300.

Join employee data with salary history.

301.

Find latest salary for every employee.

302.

Calculate salary growth percentage.

303.

Find employees whose salary increased by more than 10%.

304.

Join employee + department + job + project data.

305.

Create an employee 360° DataFrame.

306.

Perform data-quality checks.

307.

Separate valid and invalid records.

308.

Write valid records to a Delta Silver table.

309.

Create department-level Gold data.

310.

Create employee-level Gold data.

311.

Optimize the Gold tables.

312.

Create a final dataset suitable for Power BI.



Your Practice Order

Don't try all 312 randomly.

Follow this exact sequence:

LinkedIn@AshishZope

01-15 → DataFrame Basics
16-25 → Select
26-40 → Filter
41-55 → Computed Columns
56-70 → String Functions
71-80 → NULL Handling
81-90 → Sorting
91-100 → Duplicates
101-130 → Aggregations
131-150 → Dates
151-180 → Joins
181-205 → Window Functions
206-220 → Attendance
221-235 → Data Quality
236-245 → Date Scenarios
246-260 → Delta Lake
261-275 → Performance
276-290 → Interview Scenarios
291-312 → End-to-End Project

Most important for you right now

Since you're **starting from scratch**, don't look at the advanced questions yet.

Start with **1 → 100**.

For each question:

Question → write PySpark code → run it in Databricks → display() result → move to next.

When you get stuck, send me **your code**, not just the question. I'll review it and explain **what is wrong + why + the optimized PySpark approach**, so you actually build the skill rather

than memorizing solutions.

